

ANASTA-Pro

Predictive, Noise-Preserving Eigenspace Fountain Authentication Protocol Technical Design Document v1.0

June 4, 2026

Abstract

ANASTA-Pro (Adaptive Noise Aware Subspace Tiered Authentication Protocol) is a protocol designed for contested cyber-physical systems operating under electronic warfare (EW) jamming, severe resource constraints, and active adversarial injection. The protocol doesn't use the conventional Fuzzy Extractor paradigm, which erases hardware noise to force a binary Accept/Reject authentication state. Instead, we treat the hardware noise, environmental degradation, and network bit errors as information-rich signals. By partitioning a Principal Component Analysis (PCA) eigenmatrix into two subspaces: One for identity and another for sensor health health, deploying a rolling fountain bit-mask for progressive key reconstruction under degraded channels, and binding the receiver to a predictive Linear Kalman Filter state estimator, ANASTA-Pro simultaneously achieves zero-overhead hardware health monitoring, low-latency cryptographic authentication, and active injection attack detection within a single unified protocol. This document provides the complete technical specification, mathematical formulation, packet format definition, implementation architecture, and simulation validation roadmap for ANASTA-Pro.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Scope and Applicability	3
1.3	Document Organisation	3
2	System Architecture Overview	4
2.1	Architectural Layers	4
2.2	System Data Flow	4
3	Layer 1: The Subspace Separation Architecture	5
3.1	Offline Factory Enrollment	5
3.2	Dual Subspace Partition	5
3.3	Key Generation	5
3.4	Security Footprint	6

4	Layer 2: The Progressive Fountain Protocol	6
4.1	Protocol State Machine	6
4.2	Synchronised Nonce and Mask Generation	6
4.3	Tier 1: High-Fidelity Mode	7
4.4	Tier 2: Progressive Fountain Mode	7
4.5	Server-Side Fountain Reconstruction	7
5	Layer 3: The Predictive Kalman Innovation Engine	8
5.1	State Space Formulation	8
5.2	Dynamic Observation Noise Inflation	8
5.3	Kalman Filter Recursion	9
5.4	Innovation-Based Attack Detection	9
5.5	Hardware Trust Score	10
6	Packet Format Specification	10
6.1	Header Format	10
6.2	Field Definitions	10
6.3	Total Packet Footprints	11
7	Multi-Domain Operational Configuration	12
8	Security Analysis	12
8.1	Threat Model	12
8.2	Security Properties	12
8.3	Known Limitations	13
9	Implementation Architecture	13
9.1	Technology Stack	13
9.2	Module Architecture	14
9.3	Single-Script Execution Blueprint	14
10	Simulation Validation Scenarios	15
10.1	Scenario A: Natural Hardware Degradation	15
10.2	Scenario B: Electronic Warfare Barrage	16
10.3	Scenario C: Active Injection Attack	16
11	Conclusion	17

1 Introduction

1.1 Motivation

Cyber-physical systems (CPS) deployed in contested operational environments face a convergence of threats that existing authentication protocols are not designed to handle simultaneously. Tactical unmanned aerial vehicles (UAVs) operate under broadband electronic warfare jamming that degrades packet delivery ratios (PDR) below 80%. Structural health monitoring (SHM) sensor nodes are constrained by energy harvesting capacitors that may drop below 2.2V during extended operation. Battlefield Internet of Things (IoT) nodes face physical capture, side-channel memory dumps, and active Man-in-the-Middle (MitM) injection attacks.

The dominant industry response to hardware authentication under noise is the *Fuzzy Extractor* paradigm. A Fuzzy Extractor applies error-correcting codes (typically BCH or Reed-Solomon) to a noisy Physical Unclonable Function (PUF) response, forcing it into a stable binary string suitable for cryptographic key derivation. This approach has three fundamental limitations in contested CPS environments:

- (i) **Computational overhead at the edge:** BCH decoding requires thousands of clock cycles, which is prohibitive for energy-harvesting SHM nodes.
- (ii) **Information destruction:** By correcting hardware noise to a fixed reference, the Fuzzy Extractor discards the very signal that encodes hardware degradation state, making zero-overhead health monitoring impossible.
- (iii) **Static footprint:** The authentication payload size is fixed regardless of channel conditions. Under heavy jamming, large authentication packets are preferentially dropped, causing authentication failure precisely when security is most critical.

ANASTA-Pro addresses all three limitations through a fundamentally different architectural philosophy: *noise is not an obstacle to be corrected; it is a signal to be exploited.*

1.2 Scope and Applicability

This design document specifies ANASTA-Pro for three operational domains:

- **Tactical UAV:** Channel bandwidth and link budget constraints; primary threat is broadband jamming and signal exploitation.
- **Structural Health Monitoring (SHM):** Energy cache constraints; primary threat is eavesdropping and model cloning.
- **Battlefield IoT:** Processor and RAM constraints; primary threat is node stealing and side-channel memory dumps.

1.3 Document Organisation

The remainder of this document is structured as follows. Section 2 presents the high-level system architecture. Section 3 specifies the Subspace Separation Architecture. Section 4 specifies the Progressive Fountain Protocol. Section 5 specifies the Predictive Kalman

Innovation Engine. Section 6 defines the complete packet format. Section 8 provides a security analysis. Section 9 describes the Python + ns-3 implementation architecture. Section 10 defines the simulation validation scenarios.

2 System Architecture Overview

2.1 Architectural Layers

ANASTA-Pro is organised into three distinct architectural layers, each with a well-defined interface to the adjacent layers:

Layer 1 — The Hardware Layer Executes on the Edge Asset (UAV, SHM sensor, IoT node). Responsible for PCA eigenspace projection, dual-subspace key generation, and packet serialisation. Computationally minimal: single-cycle XOR masking only.

Layer 2 — The Wire Layer The communication channel, modelled as a contested wireless link subject to jamming, fading, and bit errors. Carries the ANASTA-Pro application-layer packet. Simulated using ns-3.

Layer 3 — The Server Layer Executes on the Digital Twin Server. Responsible for fountain reconstruction, Hamming Distance computation, Kalman state estimation, innovation-based attack detection, and trust score generation. Computationally unconstrained.

2.2 System Data Flow

Figure 1 illustrates the end-to-end data flow of the ANASTA-Pro protocol.

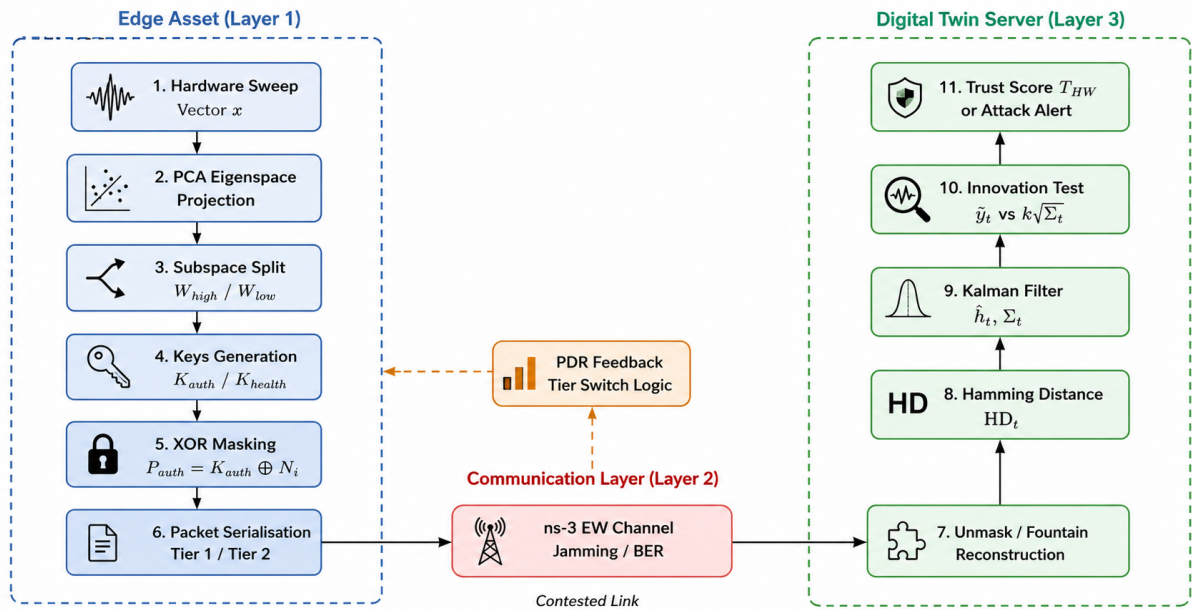


Figure 1: ANASTA-Pro end-to-end architecture.

3 Layer 1: The Subspace Separation Architecture

3.1 Offline Factory Enrollment

During factory enrollment, a set of N hardware sweep vectors $\{x_1, x_2, \dots, x_N\}$ are collected from the device under controlled conditions. Each sweep vector $x \in \mathbb{R}^d$ represents a raw physical measurement of the impedance sweep across a frequency spectrum.

The PCA projection matrix W is computed from the covariance matrix of the normalised enrollment vectors. The rows of W are the eigenvectors of the sample covariance matrix, sorted in *descending* order of their associated eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_M$.

3.2 Dual Subspace Partition

Definition 3.1 (Identity Subspace). *The identity subspace projection matrix $W_{high} \in \mathbb{R}^{128 \times d}$ is formed from the top 128 rows of W (highest eigenvalues):*

$$W_{high} = W_{1:128,:} \quad (1)$$

These components capture the maximum variance of the enrollment distribution, corresponding to the permanent microscopic imperfections of the hardware silicon that are stable against ambient thermal fluctuations.

Definition 3.2 (Health Subspace). *The health subspace projection matrix $W_{low} \in \mathbb{R}^{32 \times d}$ is formed from the bottom 32 rows of W (lowest eigenvalues):*

$$W_{low} = W_{128:160,:} \quad (2)$$

These components are hyper-sensitive to micro-structural shifts, crystal lattice degradation, device aging, and physical damage, precisely because they capture the directions of minimum variance in the healthy enrollment distribution.

Remark 3.1. *This is a deliberate inversion of the standard PCA dimensionality reduction paradigm, which discards low-variance components as uninformative. ANASTA-Pro retains and exploits the low-variance subspace as a Hardware Degradation Fingerprint (HDF).*

3.3 Key Generation

Definition 3.3 (Authentication Key). *The binary authentication key $K_{auth} \in \{0, 1\}^{128}$ is generated by:*

$$K_{auth} = \mathbf{1}(W_{high} \cdot \tilde{x} > 0) \quad (3)$$

where $\mathbf{1}(\cdot)$ denotes the element-wise indicator function (1 if positive, 0 otherwise).

Definition 3.4 (Health Degradation Vector). *The real-valued health degradation vector $K_{health} \in \mathbb{R}^{32}$ is generated by:*

$$K_{health} = W_{low} \cdot \tilde{x} \quad (4)$$

3.4 Security Footprint

The Edge Asset stores only:

- $W_{high} \in \mathbb{R}^{128 \times d}$: Identity projection matrix
- $W_{low} \in \mathbb{R}^{32 \times d}$: Health projection matrix
- $\mu \in \mathbb{R}^d$: Enrollment mean vector
- $\sigma \in \mathbb{R}^d$: Enrollment scale vector
- S_{master} : Factory master key (128-bit AES key)

The Edge Asset **never** stores the baseline reference keys $K_{auth}^{(ref)}$ or any device identifier. A complete memory dump of a captured asset reveals no cryptographic secrets that can be used to impersonate the device.

4 Layer 2: The Progressive Fountain Protocol

4.1 Protocol State Machine

ANASTA-Pro operates as a two-tier state machine. The active tier is determined by real-time channel quality metrics delivered via the feedback channel from the Digital Twin Server.

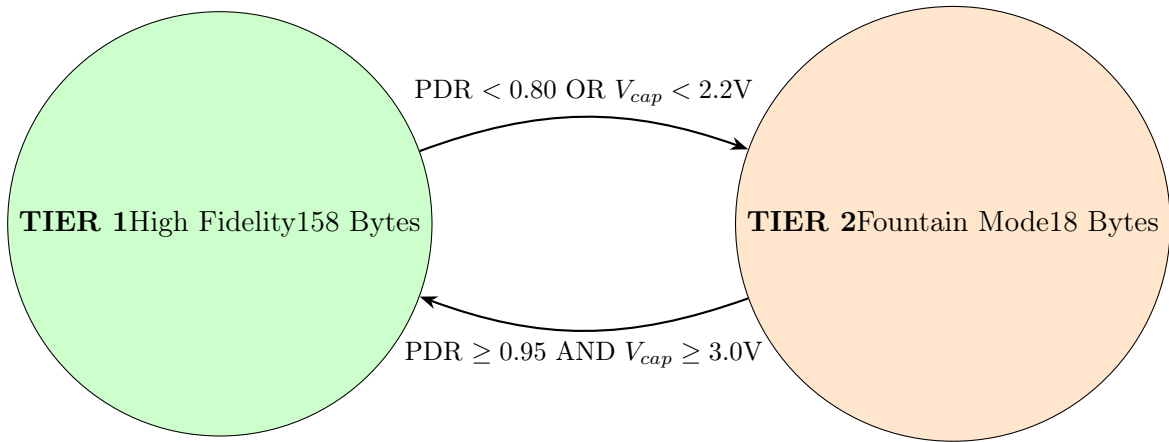


Figure 2: ANASTA-Pro two-tier protocol state machine.

4.2 Synchronised Nonce and Mask Generation

At transmission index i , both the Edge Node and the Digital Twin Server independently derive two values from the shared factory master key S_{master} using AES in Counter Mode (AES-CTR):

$$N_i = \text{AES-CTR}(S_{master}; \text{ctr} = i) \in \{0, 1\}^{128} \quad (5)$$

$$S_i = \text{AES-CTR}(S_{master} \mid \text{SALT}; \text{ctr} = i) \in \{0, 1\}^{32} \quad (6)$$

where $\mid$ denotes concatenation and SALT is a device-specific constant stored at enrollment.

Remark 4.1. *AES-CTR is used rather than a non-cryptographic PRNG (e.g., Mersenne Twister) because the security of the masking scheme depends on the unpredictability of N_i and S_i to an adversary who does not possess S_{master} . A non-cryptographic PRNG whose internal state can be reconstructed from observed outputs would completely break the authentication scheme.*

4.3 Tier 1: High-Fidelity Mode

Tier 1 is active when the communication link is clear ($PDR \geq 0.95$) or when a structural anomaly demands immediate deep diagnostics.

Definition 4.1 (Tier 1 Authentication Payload).

$$P_{auth}^{(1)} = K_{auth} \oplus N_i \in \{0, 1\}^{128} \quad (7)$$

Definition 4.2 (Tier 1 Diagnostic Payload). *The raw health vector is appended directly:*

$$P_{health}^{(1)} = K_{health} \in \mathbb{R}^{32} \quad (8)$$

serialised as 32 IEEE 754 single-precision floats (128 bytes).

Tier 1 Security Overhead: $16 + 128 = 144$ bytes of payload, plus 14 bytes of header, for a total packet footprint of **158 bytes**.

4.4 Tier 2: Progressive Fountain Mode

Tier 2 is triggered when $PDR < 0.80$ or $V_{cap} < 2.2V$. Diagnostic transmission is suspended to conserve bandwidth and sensor sweep power.

Definition 4.3 (Fountain Compression Operator). *Given a 128-bit key K and a 32-bit seed S_i , the fountain compression operator $Compress_{32}(K, S_i)$ uses S_i to deterministically select 32 distinct bit indices $\mathcal{I}_i \subset \{0, 1, \dots, 127\}$ ($|\mathcal{I}_i| = 32$) and extracts the corresponding bits:*

$$Compress_{32}(K, S_i) = \{K_j : j \in \mathcal{I}_i\} \in \{0, 1\}^{32} \quad (9)$$

The index set $\mathcal{I}_i$ is generated by seeding a deterministic selection algorithm with S_i and drawing 32 samples without replacement from $\{0, \dots, 127\}$.

Definition 4.4 (Tier 2 Fountain Payload).

$$P_{fountain}^{(2)} = Compress_{32}(K_{auth}, S_i) \oplus Compress_{32}(N_i, S_i) \in \{0, 1\}^{32} \quad (10)$$

Tier 2 Security Overhead: 4 bytes of payload, plus 14 bytes of header, for a total packet footprint of **18 bytes** — a 97.2% reduction compared to Tier 1.

4.5 Server-Side Fountain Reconstruction

The Digital Twin Server maintains a rolling reconstruction matrix $\mathbf{R} \in \{0, 1, ?\}^{128}$, where ‘?’ denotes an unobserved bit position. Upon receiving packet i in Tier 2 mode:

1. Compute S_i from eq. (6).

2. Derive $\mathcal{I}_i$ from S_i .
3. Unmask: $\hat{K}_{auth}[\mathcal{I}_i] = P_{fountain}^{(2)} \oplus \text{Compress}_{32}(N_i, S_i)$.
4. Update $\mathbf{R}[\mathcal{I}_i]$ with the recovered bits.
5. If all 128 positions of $\mathbf{R}$ are observed, reconstruction is complete.

Theorem 4.1 (Expected Reconstruction Convergence). *Under uniform random index selection without replacement, the expected number of packets $\mathbb{E}[T]$ required to observe all 128 bit positions is given by the coupon collector problem with groups of size 32:*

$$\mathbb{E}[T] = \frac{128}{32} \cdot H\left(\frac{128}{32}\right) \approx 4\text{-}6 \text{ packets} \quad (11)$$

where $H(\cdot)$ denotes the harmonic number, and the approximation accounts for overlap between successive index sets.

5 Layer 3: The Predictive Kalman Innovation Engine

5.1 State Space Formulation

The Digital Twin Server models the true underlying physical degradation of the device as a hidden state $h_t \in \mathbb{R}$ evolving according to a linear Gaussian state space model.

Definition 5.1 (State Transition Model).

$$h_t = h_{t-1} + \alpha \Delta t + w_t, \quad w_t \sim \mathcal{N}(0, Q) \quad (12)$$

where:

- $\alpha \geq 0$ is the physical aging coefficient of the device (units: HD/second)
- Δt is the inter-packet time interval
- $Q > 0$ is the process noise covariance, encoding the volatility of the physical aging process

Definition 5.2 (Observation Model).

$$HD_t = h_t + v_t, \quad v_t \sim \mathcal{N}(0, R_t) \quad (13)$$

where HD_t is the measured Hamming Distance between the received key and the factory-enrolled reference key, and R_t is the dynamic observation noise covariance.

5.2 Dynamic Observation Noise Inflation

The observation noise covariance R_t is not fixed. It is inflated dynamically based on network-layer telemetry delivered by the ns-3 simulation layer:

$$R_t = R_{base} \cdot f_{BER}(\text{BER}_t) \cdot f_{SNR}(\text{SNR}_t) \cdot f_{PDR}(\text{PDR}_t) \quad (14)$$

where the inflation factors are defined as:

$$f_{BER}(\text{BER}_t) = \exp(10 \cdot \text{BER}_t) \quad (15)$$

$$f_{SNR}(\text{SNR}_t) = \frac{1}{1 + \log(1 + 10^{\text{SNR}_t/10})} \quad (16)$$

$$f_{PDR}(\text{PDR}_t) = 1 + 5(1 - \text{PDR}_t) \quad (17)$$

This coupling between the physical communication environment and the cryptographic trust assessment prevents false-positive attack alerts during periods of legitimate channel degradation.

5.3 Kalman Filter Recursion

The standard Linear Kalman Filter recursion is applied at each time step t :

Prediction Step:

$$\hat{h}_{t|t-1} = \hat{h}_{t-1|t-1} + \alpha \Delta t \quad (18)$$

$$\Sigma_{t|t-1} = \Sigma_{t-1|t-1} + Q \quad (19)$$

Innovation:

$$\tilde{y}_t = HD_t - \hat{h}_{t|t-1} \quad (20)$$

Innovation Covariance:

$$S_t = \Sigma_{t|t-1} + R_t \quad (21)$$

Kalman Gain:

$$K_t = \frac{\Sigma_{t|t-1}}{S_t} \quad (22)$$

Update Step (executed only if no attack detected):

$$\hat{h}_{t|t} = \hat{h}_{t|t-1} + K_t \tilde{y}_t \quad (23)$$

$$\Sigma_{t|t} = (1 - K_t) \Sigma_{t|t-1} \quad (24)$$

5.4 Innovation-Based Attack Detection

Definition 5.3 (Structural Validation Threshold). *The innovation threshold at time t is:*

$$\tau_t = k \cdot \sqrt{S_t} \quad (25)$$

where $k > 0$ is a tunable sensitivity parameter. For a Gaussian innovation sequence under the null hypothesis of normal aging, $k = 3$ corresponds to a 99.7% confidence interval.

Definition 5.4 (Attack Detection Rule).

$$\text{Decision}_t = \begin{cases} \text{NORMAL} & \text{if } |\tilde{y}_t| \leq \tau_t \\ \text{ATTACK} & \text{if } |\tilde{y}_t| > \tau_t \end{cases} \quad (26)$$

If $\text{Decision}_t = \text{ATTACK}$:

- The Kalman update step is **skipped** (the state estimate is not corrupted by the malicious observation).
- The trust score is set to $T_{HW} = 0.00$.
- The packet is dropped and an alert is raised.
- The edge asset is commanded to downshift to Tier 2 immediately.

5.5 Hardware Trust Score

When no attack is detected, the continuous Hardware Trust Score $T_{HW} \in [0, 1]$ is computed as:

$$T_{HW,t} = \exp(-\hat{h}_{t|t}) \cdot \frac{1}{1 + \Sigma_{t|t}} \quad (27)$$

The first factor decays exponentially with estimated physical degradation. The second factor penalises high estimation uncertainty.

6 Packet Format Specification

6.1 Header Format

All ANASTA-Pro packets share a common 14-byte application-layer header.

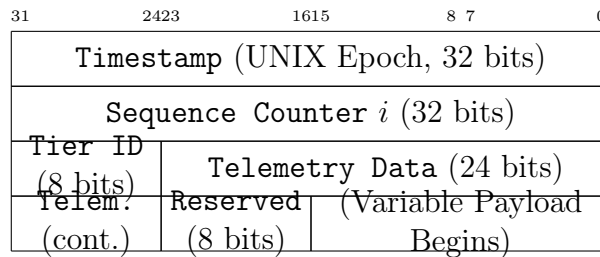


Figure 3: ANASTA-Pro common header format (14 bytes).

6.2 Field Definitions

Field	Bits	Bytes	Description
Timestamp	32	4	UNIX epoch timestamp. In asynchronous SHM meshes with clock drift, this field is zero-padded and execution is delegated entirely to the Sequence Counter.
Sequence Counter	32	4	Strictly monotonic, unsigned rolling integer. Packets with sequence numbers $\leq$ the last accepted sequence number are immediately dropped at the application boundary, providing replay attack resistance.
Tier ID	8	1	Structure enforcement bitflag. 0x01 = Tier 1 (High-Fidelity Mode); 0x02 = Tier 2 (Fountain Mode).

Field	Bits	Bytes	Description
Telemetry Data	32	4	Core cyber-sensor values: UAV flight velocity, pipeline flow rate, or structural frame strain metric. Encoded as IEEE 754 single-precision float.
Reserved	8	1	Null spacer padding ensuring explicit 32-bit compilation block alignment.
Auth Payload (T1)	128	16	Masked authentication key $P_{auth}^{(1)} = K_{auth} \oplus N_i$.
Health Payload (T1)	1024	128	Raw health vector K_{health} serialised as 32 IEEE 754 single-precision floats.
Fountain Payload (T2)	32	4	Fountain-masked 32-bit segment $P_{fountain}^{(2)}$.

Table 1: ANASTA-Pro packet field definitions.

6.3 Total Packet Footprints

Mode	Header	Payload	Total
Tier 1 (High-Fidelity)	14 B	144 B	158 B
Tier 2 (Fountain)	14 B	4 B	18 B
Footprint reduction (Tier 2 vs Tier 1)			88.6%
Payload reduction (Tier 2 vs Tier 1)			97.2%

Table 2: ANASTA-Pro packet footprint comparison.

7 Multi-Domain Operational Configuration

Domain	Primary Resource Limit	Dominant Threat	Tier 1 Entry Criteria	Tier 2 Trigger
Tactical UAV	Channel bandwidth / link budget	Broadband jamming / signal exploitation	$PDR \geq 0.95$ over rolling 20 steps	$PDR < 0.80$ OR SNR collapse
Structural Health (SHM)	Energy caches / storage caps	Eavesdropping / model cloning	$V_{cap} \geq 3.0V$ AND anomaly flags clear	$V_{cap} < 2.2V$ OR data criticality = baseline
Battlefield IoT	Processor cycles / RAM	Node stealing / side-channel dump	Active security audit call	Tx buffer saturation OR routing queue bottleneck

Table 3: Multi-domain operational configuration parameters.

8 Security Analysis

8.1 Threat Model

ANASTA-Pro assumes the following adversary capabilities:

- **Passive eavesdropping:** The adversary can observe all packets transmitted over the wireless channel.
- **Active injection:** The adversary can inject arbitrary packets into the channel.
- **Replay:** The adversary can retransmit previously observed packets.
- **Physical capture:** The adversary can perform a complete memory dump of the Edge Asset.
- **Computational:** The adversary is computationally bounded (cannot break AES-128).

8.2 Security Properties

Replay Attack Resistance The strictly monotonic Sequence Counter ensures that any replayed packet (with sequence number $\leq$ the last accepted value) is immediately dropped at the application boundary.

Eavesdropping Resistance The authentication payload $P_{auth}^{(1)} = K_{auth} \oplus N_i$ is a one-time-pad masked ciphertext. Since N_i is derived from AES-CTR with a secret master key, an eavesdropper who observes $P_{auth}^{(1)}$ cannot recover K_{auth} without knowledge of S_{master} .

Physical Capture Resistance The Edge Asset stores no baseline reference keys or device identifiers. A complete memory dump reveals only the projection matrices $\{W_{high}, W_{low}\}$, calibration vectors $\{\mu, \sigma\}$, and the master key S_{master} . The projection matrices are device-specific but do not constitute authentication credentials without the physical hardware to generate x .

Injection Attack Detection The Kalman innovation engine detects injected packets whose Hamming Distance deviates from the predicted physical aging trajectory by more than $k\sqrt{S_t}$. Physical hardware cannot age by a large fraction of its Hamming Distance in a single inter-packet interval; such deviations are mathematically impossible under the physical aging model and are therefore flagged as attacks.

Forward Security The AES-CTR counter mode ensures that compromise of N_i for a given i does not reveal N_j for $j \neq i$, provided S_{master} remains secret.

8.3 Known Limitations

- The linear aging model (eq. (12)) is a simplification. Real hardware aging may exhibit non-linear, device-specific characteristics. The parameters α and Q require empirical calibration.
- The security of the fountain reconstruction mechanism against an adversary who can adaptively choose which packets to corrupt (rather than uniformly random corruption) requires further formal analysis.
- The protocol does not provide mutual authentication: the Edge Asset authenticates to the Digital Twin Server, but not vice versa. A compromised server could issue false tier-switch commands.

9 Implementation Architecture

9.1 Technology Stack

Component	Library	Justification
PCA Eigenspace	<code>scikit-learn</code>	Numerically stable SVD backend
Kalman Filter	<code>filterpy</code>	Purpose-built KF/UKF/EKF library
AES-CTR PRNG	<code>pycryptodome</code>	FIPS-compliant AES implementation
Packet Serialisation	<code>struct</code>	Zero-dependency binary packing
Network Simulation	<code>ns-3</code> (Python bindings)	Native ns-3 pybindgen interface
Data Ingestion	<code>pandas, numpy</code>	CSV parsing and matrix operations
Visualisation	<code>matplotlib</code>	Publication-ready figure generation

Table 4: ANASTA-Pro Python implementation technology stack.

9.2 Module Architecture

The implementation is organised into four Python modules:

`enrollment.py` Offline factory enrollment: loads hardware sweep CSVs, computes μ , σ , fits PCA, partitions W_{high} and W_{low} , serialises enrollment artefacts.

`edge_module.py` Implements `AnastaEdgeApp` as an `ns.network.Application` subclass. Generates K_{auth} , K_{health} , applies AES-CTR masking, serialises packets, implements tier state machine.

`server_module.py` Implements `DigitalTwinServerApp` as an `ns.network.Application` subclass. Performs fountain reconstruction, Hamming Distance computation, Kalman filter recursion, innovation test, trust score computation.

`run_simulation.py` Top-level simulation entry point. Initialises ns-3 simulator clock, constructs node topology, installs applications, runs deterministic simulation loop, exports results.

9.3 Single-Script Execution Blueprint

```

1 import ns.core
2 import ns.network
3 import ns.internet
4 import ns.wifi
5 import ns.mobility
6 from edge_module import AnastaEdgeApp
7 from server_module import DigitalTwinServerApp
8
9 def main():
10     # 1. Parse command-line arguments
11     cmd = ns.core.CommandLine()
12     cmd.Parse()
13
14     # 2. Create simulation nodes
15     nodes = ns.network.NodeContainer()
16     nodes.Create(3) # UAV, BaseStation, Jammer
17
18     # 3. Configure wireless channel
19     wifi = ns.wifi.WifiHelper()
20     wifi.SetStandard(ns.wifi.WIFI_STANDARD_80211n)
21
22     channel = ns.wifi.YansWifiChannelHelper.Default()
23     channel.AddPropagationLoss(
24         "ns3::LogDistancePropagationLossModel"
25     )
26     phy = ns.wifi.YansWifiPhyHelper()
27     phy.SetChannel(channel.Create())
28
29     # 4. Configure mobility
30     mobility = ns.mobility.MobilityHelper()
31     mobility.SetMobilityModel(
32         "ns3::ConstantVelocityMobilityModel"
33     )
34     mobility.Install(nodes)

```

```

35
36 # Set UAV velocity vector (flies into jammer radius)
37 uav_node = nodes.Get(0)
38 uav_mob = uav_node.GetObject(
39     ns.mobility.ConstantVelocityMobilityModel.GetTypeId()
40 )
41 uav_mob.SetVelocity(ns.core.Vector(5.0, 0.0, 0.0))
42
43 # 5. Install ANASTA-Pro applications
44 edge_app = AnastaEdgeApp(
45     master_key=b'\x00' * 16, # Replace with real key
46     tier=1,
47     seq_start=0
48 )
49 edge_app.SetStartTime(ns.core.Seconds(1.0))
50 edge_app.SetStopTime(ns.core.Seconds(59.0))
51 nodes.Get(0).AddApplication(edge_app)
52
53 server_app = DigitalTwinServerApp(
54     master_key=b'\x00' * 16,
55     Q_process=0.01,
56     R_base=1.0,
57     k_threshold=3.0
58 )
59 server_app.SetStartTime(ns.core.Seconds(0.0))
60 server_app.SetStopTime(ns.core.Seconds(60.0))
61 nodes.Get(1).AddApplication(server_app)
62
63 # 6. Run simulation
64 ns.core.Simulator.Stop(ns.core.Seconds(60.0))
65 ns.core.Simulator.Run()
66 ns.core.Simulator.Destroy()
67
68 # 7. Export results
69 server_app.export_results("results/")
70
71 if __name__ == "__main__":
72     main()

```

Listing 1: Top-level simulation entry point (run_simulation.py).

10 Simulation Validation Scenarios

10.1 Scenario A: Natural Hardware Degradation

Objective: Demonstrate that the Kalman filter correctly tracks gradual hardware aging without triggering false-positive attack alerts.

Configuration:

- Mild jamming: PDR = 0.90, BER = 0.001
- Synthetic aging: $\alpha = 0.005$ HD/s, $Q = 0.01$
- Simulation duration: 60 seconds
- No injected attacks

Expected Outputs:

- T_{HW} decreases smoothly from 1.0 to approximately 0.7 over 60 seconds
- Zero attack alerts raised
- Protocol remains in Tier 1 throughout

Publication Figure: Time series plot of T_{HW} , $\hat{h}_t$, and HD_t with $\pm k\sqrt{S_t}$ confidence bands.

10.2 Scenario B: Electronic Warfare Barrage

Objective: Demonstrate that the fountain protocol survives heavy jamming by downshifting to Tier 2 and maintaining authentication continuity.

Configuration:

- UAV flies at 5 m/s directly into jammer radius
- PDR degrades from 0.98 to 0.45 over 30 seconds
- Jammer activates at $t = 15$ s
- Tier 2 trigger: $PDR < 0.80$

Expected Outputs:

- Protocol downshifts to Tier 2 at approximately $t = 20$ s
- Packet footprint drops from 158 B to 18 B
- Full 128-bit key reconstruction achieved within 4–6 Tier 2 packets
- Authentication continuity maintained throughout the barrage

Publication Figure: Dual-axis plot of PDR (left axis) and packet size in bytes (right axis) versus simulation time, with tier transition marked.

10.3 Scenario C: Active Injection Attack

Objective: Demonstrate that the Kalman innovation engine detects and rejects a malicious replay/injection attack.

Configuration:

- Stable channel: $PDR = 0.95$, $BER = 0.0001$
- Mild aging: $\alpha = 0.002$ HD/s
- Injection event at $t = 30$ s: adversary injects a packet with $HD = 45$ (vs. predicted $\hat{h}_{30} \approx 8$)

Expected Outputs:

- Innovation spike at $t = 30$ s: $|\tilde{y}_{30}| \gg k\sqrt{S_{30}}$

- Attack alert raised; packet dropped
- Kalman state not updated; $\hat{h}_t$ continues on correct trajectory
- T_{HW} drops to 0.00 at $t = 30s$ and recovers on subsequent legitimate packets

Publication Figure: Time series of $\tilde{y}_t$ with $\pm\tau_t$ bounds, with injection event marked by a vertical dashed line.

11 Conclusion

This document has presented the complete technical specification of ANASTA-Pro, a novel authentication protocol for contested cyber-physical systems. The protocol’s three primary architectural innovations are:

1. **Dual-purpose eigenspace decomposition:** The semantic reinterpretation of the PCA eigenvalue spectrum, assigning the high-variance subspace to stable identity authentication and the low-variance subspace to hardware health monitoring, constitutes a conceptual inversion of the standard PCA dimensionality reduction paradigm.
2. **Progressive fountain key reconstruction:** The deterministic, cryptographically seeded fountain-style bit-selection mechanism achieves full 128-bit key reconstruction within 4–6 packets over a 4-byte per-packet channel footprint, enabling authentication continuity under severe jamming.
3. **Physics-constrained injection detection:** The Kalman innovation engine provides a mathematically principled criterion for distinguishing legitimate hardware aging from adversarial injection, with dynamic noise inflation preventing false positives during legitimate channel degradation.

The Python + ns-3 simulation framework described in section 9 provides a credible and reproducible validation platform for demonstrating these properties across the three evaluation scenarios defined in section 10.